

FindEZ — local LLM evaluation pack

For testing whether an open-weight model can replace OpenAI. Written 2026-08-09.

Everything here is copied from the live code in `backend/app/services/openai_service.py` and `ai_agent.py` — these are the actual prompts and schemas in production, not approximations.

TL;DR — read this bit if nothing else

1. **No sample images needed.** Every scanned photo is already stored with the items GPT-4o extracted from it. One SQL query (§ *You already have a labelled test set*) gives you an eval set with labels. Treat GPT-4o's output as the **baseline to beat, not ground truth** — some of it is visibly poor.
 2. **Three jobs, not one.** Single-item extraction (easy), multi-item extraction (moderate), and the **conversational agent** (hard). The agent turns *"add 20 M3×10 screws to the Workshop"* into a database write via tool calling. **That's the real test** — a model that fails there deletes the wrong row rather than merely answering badly.
 3. **100% valid JSON is mandatory.** Every call is a forced function call, and production has no fallback if one comes back malformed. Anything under 100% is disqualifying.
 4. **Hallucinated items are worse than missed ones** — they put phantom stock in a team's inventory, which nobody notices until it matters.
 5. **Don't evaluate on fasteners.** M3×8 vs M3×10 is a 2mm difference and no vision model can recover absolute length from an uncalibrated photo. Nothing passes, GPT-4o included. Test on visually distinctive parts: motors, servos, hubs, wheels, sensors.
 6. **Integration is config, not code.** Serve behind an OpenAI-compatible endpoint (vLLM, SGLang) and `openai_service.py` needs a base URL change, nothing more.
 7. **Where to start:** Qwen-VL family first (strongest open weights on OCR/document work), InternVL3 if MIT licensing matters, Pixtral/Molmo for Apache-2.0. Note that the best flat-text OCR model is often *not* the best at schema-constrained JSON — grade on JSON validity and field accuracy only.
-

The important thing first: you already have a labelled test set

You don't need me to invent sample pictures. **Every photo a user has scanned is stored, and the items GPT-4o extracted from it are sitting in your database.** That's an eval set with labels, built from real usage.

Pull it with:

```
SELECT i.image_url,
       i.name, i.category, i.subcategory, i.brand, i.part_number,
       i.quantity, i.confidence, i.location
FROM items i
WHERE i.image_url IS NOT NULL AND TRIM(i.image_url) <> ''
ORDER BY i.image_url, i.name;
```

Group the rows by `image_url` and each group is one test case: an image, and the list of items the current system produced from it.

Important caveat: this is a baseline, not ground truth. GPT-4o's output is what you have today, not what's correct. Some of it is visibly weak — one real row reads `White Rectangular Object (likely Menu or Booklet)`. The practical bar is **parity or better**, and in places that's a low bar.

For anything you plan to publish numbers about, have a human correct the labels first.

FindEZ asks a model to do three separate jobs

They have very different difficulty, and a model can pass one and fail another.

Job	What it does	Difficulty for open weights
1. Single-item extraction	One photo → one item's fields	Easy
2. Multi-item extraction	One photo of a shelf → every item in it	Moderate
3. Conversational agent	Natural language → database writes, via tool calls	Hard — this is the real test

Job 3 is where open models usually fall over, and it's the one that can corrupt data rather than just be unhelpful. Test it before deciding anything.

Job 1 — single-item extraction

Model called with: an image plus a forced function call. Temperature is left default.

System prompt (verbatim):

```
You extract inventory fields. If uncertain, make best effort and keep strings short.
```

User message: the image (base64 data URL, `detail: high`) plus the text:

```
Extract inventory fields from this image.
```

Tool schema (verbatim):

```
{
  "name": "extract_inventory_fields",
  "description": "Extract structured inventory fields from an image of an item, receipt, or barcode label.",
  "parameters": {
    "type": "object",
    "properties": {
      "name": { "type": "string" },
      "category": { "type": "string" },
      "quantity": { "type": "integer" },
      "location": { "type": "string" },
      "barcode": { "type": ["string", "null"] },
      "purchase_source": { "type": ["string", "null"] },
      "notes": { "type": ["string", "null"] }
    },
    "required": ["name", "category", "quantity", "location"],
    "additionalProperties": false
  }
}
```

`tool_choice` is forced to this function, so the model has no option to reply in prose.

Expected output shape:

```
{
  "name": "Raspberry Pi 4B 4GB",
  "category": "Electronics",
  "quantity": 1,
  "location": "Workshop",
  "barcode": null,
  "purchase_source": null,
  "notes": null
}
```

Pass criteria:

- Valid JSON matching the schema, **every time** — a malformed call is a hard failure, since production has no fallback path
- `name` specific enough to be searchable ("Raspberry Pi 4B 4GB", not "circuit board")
- `quantity` a sensible integer
- All four required fields present

Job 2 — multi-item extraction

This is the harder vision task: a photo of a shelf or bin, and the model must enumerate everything.

System prompt (verbatim):

You are an expert inventory scanner with the precision of a professional home organizer and the knowledge of a product database. Your job is to identify and extract EVERY single item visible in an image – nothing is too small or too obvious to include.

RULES:

- Identify ALL items in the image, even partially visible ones
- Never skip background items, items on shelves, items behind other items, or items that seem minor
- For each item, extract: exact product name, brand (if visible), quantity (count carefully), category, and any text visible on packaging
- If you see a box or container, identify what it is AND what it likely contains if labeled
- For food items: include flavor, size, variant (e.g. 'Lay's Classic Chips 8oz' not just 'chips')
- For electronics: include model number or generation if visible
- For cleaning/household products: include the full product name and size
- For books: include full title and author if visible
- If quantity is ambiguous, err on the side of counting more carefully – look for multiples
- Never group items together – each distinct product is its own entry
- Confidence score: only mark as low confidence if the item is truly unidentifiable

User message: image (**detail: high**) plus:

Scan this image with maximum thoroughness. Extract EVERY item you can see. Be exhaustive – I would rather have too many items than miss any. For each item provide: name (specific, not generic), brand, quantity, category (one of: Food, Electronics, Clothing, Health, Home, Office, Supplies, Toys, Cosmetics, Other), and confidence (0.0-1.0). Return as structured JSON array.

Tool schema (verbatim):

```

{
  "name": "extract_inventory_items",
  "description": "Detect multiple inventory items in an image and return structured fields for each detected item.",
  "parameters": {
    "type": "object",
    "properties": {
      "items": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "name": { "type": "string" },
            "category": { "type": "string" },
            "subcategory": { "type": ["string", "null"] },
            "quantity": { "type": "integer" },
            "location": { "type": ["string", "null"] },
            "brand": { "type": ["string", "null"] },
            "part_number": { "type": ["string", "null"] },
            "barcode": { "type": ["string", "null"] },
            "tags": { "type": ["array", "null"], "items": { "type": "string" } },
            "confidence": { "type": ["number", "null"] },
            "notes": { "type": ["string", "null"] }
          },
          "required": ["name", "category", "quantity"],
          "additionalProperties": false
        }
      },
      "summary": {
        "type": "object",
        "properties": {
          "total_detected": { "type": "integer" },
          "categories": { "type": "object" }
        },
        "required": ["total_detected", "categories"],
        "additionalProperties": false
      }
    },
    "required": ["items", "summary"],
    "additionalProperties": false
  }
}

```

Real example from production. A photo of a bar table produced these five items:

```
{
  "items": [
    { "name": "Espresso Martini", "category": "Food", "quantity": 1 },
    { "name": "Golden Coaster", "category": "Home", "quantity": 1 },
    { "name": "Menu", "category": "Other", "quantity": 1 },
    { "name": "Table", "category": "Home", "quantity": 1 },
    { "name": "White Rectangular Object (likely Menu or Booklet)",
      "category": "Other", "quantity": 1 }
  ],
  "summary": { "total_detected": 5, "categories": { "Food": 1, "Home": 2, "Other": 2 } }
}
```

Note the last entry is a duplicate of "Menu" and the name is unusable. **A model that returns four clean items here is better than the incumbent, not worse.** Don't grade against GPT-4o as if it were correct.

Pass criteria:

- Valid JSON every time, no truncation on 15–20 item images
- Recall: finds most of what a human would list
- Precision: doesn't invent items that aren't there — **a hallucinated item is worse than a missed one**, because it puts phantom stock in someone's inventory
- No duplicates of the same physical object under two names
- **category** drawn from the allowed list

Job 3 — the conversational agent (the one that matters)

The chat home screen does full CRUD by natural language: *"add 20 M3×10 screws to the fastener bin"* must become a database write with the right values in the right fields. This is **tool calling**, and it's where open models most often fail — malformed calls, wrong function, wrong argument types.

The tool definitions live in `backend/app/services/ai_agent.py` (search for `'type': 'function'`). The main ones are add / update / delete item, search items, and create space.

Test cases worth running, in rising difficulty:

1. *"do I have a robot part?"* → search only, no writes
2. *"add 20 M3x10 screws to the Workshop"* → one add, quantity 20, location Workshop
3. *"I used 5 of them"* → update the *previously referenced* item, quantity –5
4. *"move everything from Garage to Workshop"* → multiple updates, correct scoping
5. *"delete the mini fridge"* → one delete, correct item, nothing else touched

Grade harshly on 3, 4 and 5. Referring back to a previous item, multi-item operations, and deletions are exactly where a weak model does real damage. A model that deletes the wrong row is disqualified regardless of how well it scores on vision.

Also test the empty-input case. There is a known production bug — documented in `CLAUDE.md` — where an empty message with a forced tool call caused the model to invent filter values and wipe the visible inventory. Any replacement model must be checked against blank and near-blank input.

Two things not to waste time on

Fasteners cannot be identified from photographs. M3×8 and M3×10 differ by 2mm and no vision model can recover absolute length from an uncalibrated image. This is a product decision already made — photo extraction is scoped to visually distinctive parts (motors, servos, hubs, wheels, sensors). Don't evaluate on screws and conclude the model is bad; nothing passes that test.

Barcode lookup isn't a model problem. It's a chain of external UPC databases, and those miss robotics SKUs almost entirely. Replacing the model changes nothing here.

Where to start on model choice

As of mid-2026 the open-weight landscape for this kind of work:

- **Qwen-VL family** — currently the strongest open weights on document and OCR benchmarks, and the usual first stop for schema-constrained extraction
- **InternVL3** — the strongest MIT-licensed option
- **Pixtral / Molmo** — Apache-2.0, strong instruction following
- **Llama 4 multimodal** — leads some general vision benchmarks
- **Granite 4.0 3B Vision** — small and specifically tuned for key-value extraction from documents

One finding worth knowing before you start: **the model that wins at flat-text OCR is often not the one that wins at schema-constrained JSON extraction** — those rank differently. Since FindEZ only ever wants structured JSON, grade on JSON validity and field accuracy, not on transcription quality.

Serve behind an OpenAI-compatible endpoint (vLLM, SGLang or similar) and the backend change is configuration rather than code — `openai_service.py` already talks to an OpenAI-shaped API.

Suggested scoring sheet

Per image, per model:

Metric	How to measure
JSON validity	% of calls returning schema-valid output. Must be 100%
Item recall	detected ÷ human-labelled
Item precision	correct ÷ detected (hallucinations count against)
Name usability	would this string be findable by search? human yes/no
Category accuracy	% assigned to the right allowed category
Latency	seconds per image at your batch size
Tool-call accuracy	% of agent test cases producing the correct call and arguments

Anything below 100% on JSON validity is disqualifying, because production has no fallback path when a tool call comes back malformed.

Sources

- [Open-source VLM guide 2026 — BentoML](#)
- [Best open-weight VLMs 2026 — Presenc](#)
- [Field guide to visual document processing 2026 — John Snow Labs](#)